

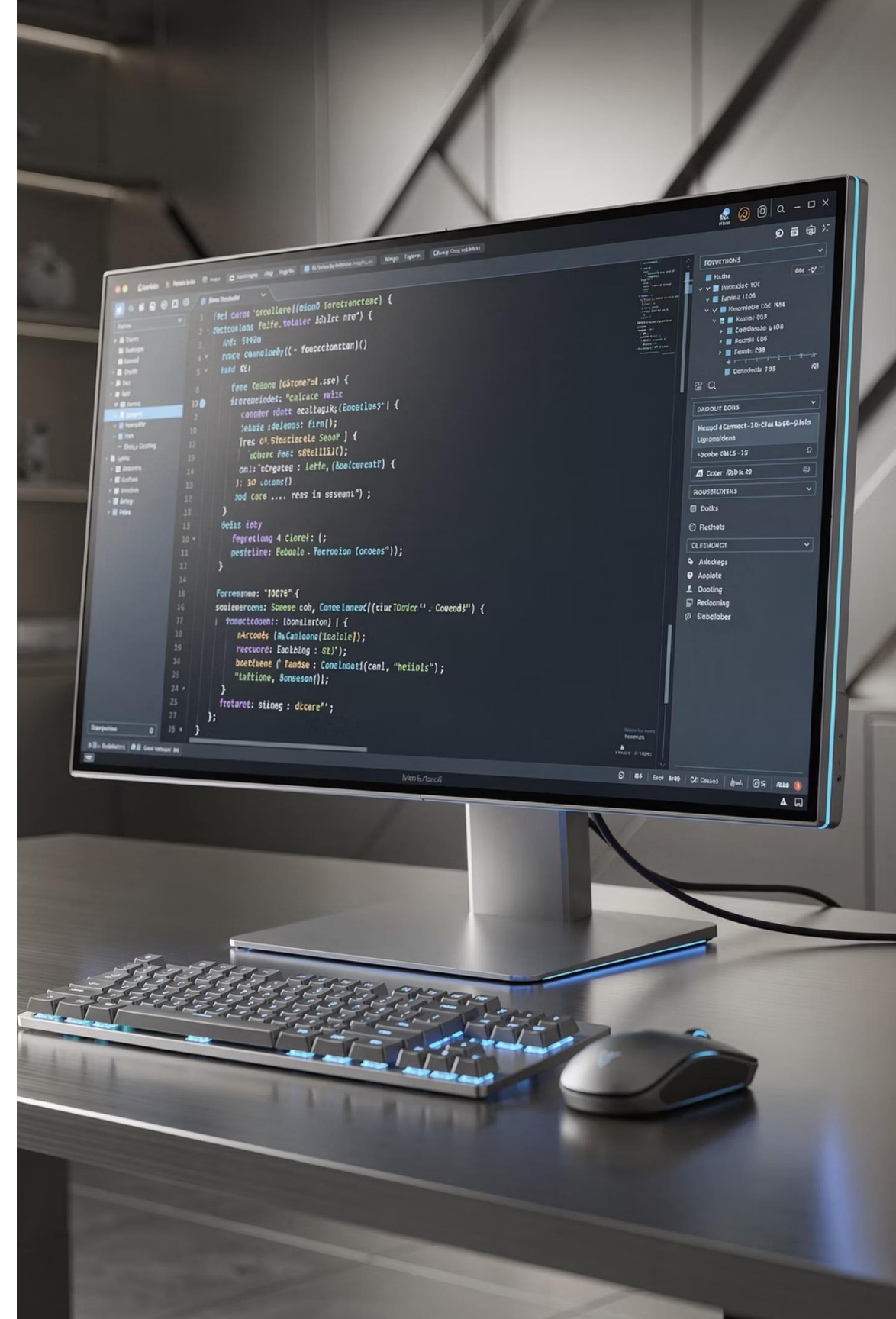
Angular Moderno

03 - Entendendo o Angular Moderno

Entendendo o Angular Moderno

OBJETIVO DA AULA

Nesta aula, você irá compreender os principais conceitos do Angular moderno, entender a diferença em relação ao modelo antigo, conhecer a abordagem Standalone e visualizar como a aplicação do curso será estruturada.



Angular Moderno

03 - Entendendo o Angular Moderno

Uma visão clara e objetiva sobre como o Angular evoluiu, o que mudou na prática e como vamos estruturar nosso projeto ao longo do curso.



O que você vai aprender nesta aula

Esta aula é a base conceitual do curso. Antes de escrever código, é fundamental entender o contexto e a estrutura do Angular moderno.



O que mudou no Angular moderno

Compreender a evolução da plataforma e por que ela ficou mais simples e produtiva.



Standalone Components

Entender o novo modelo de componentes independentes, sem necessidade de módulos.



Estrutura baseada em features

Conhecer a forma moderna de organizar projetos Angular por funcionalidade.



Visão geral do projeto final

Visualizar o e-commerce que construiremos juntos ao longo do curso.

O Angular evoluiu

Antes

O Angular era poderoso, mas complexo !

Exigia muito mais configuração e tinha uma curva de aprendizado íngreme, especialmente para quem estava começando.

Hoje

O Angular moderno (v14+, com destaque para a v17 em diante) adotou uma abordagem mais simples, moderna e eficiente. As mudanças tornam o desenvolvimento mais ágil e o código mais legível.

- Menos boilerplate
- Componentes independentes
- Melhor experiência para iniciantes
- Mais próximo das tendências do mercado



Angular Antigo — O Modelo Tradicional

Por muitos anos, o Angular exigiu o uso de **NgModules** — uma camada adicional de organização que agrupava componentes, serviços e dependências. Embora poderoso, esse modelo trazia desafios reais:

Uso obrigatório de NgModules

Todo componente precisava ser declarado dentro de um módulo. Esquecer essa etapa causava erros difíceis de diagnosticar para iniciantes.

Estrutura mais complexa

Projetos cresciam rapidamente em número de arquivos, com módulos, módulos de roteamento, módulos compartilhados e muito mais.

Curva de aprendizado maior

Entender o sistema de injeção de dependência, decorators e a hierarquia de módulos exigia bastante estudo antes de criar algo funcional.

Angular Moderno – Uma Nova Abordagem

O Angular moderno simplificou radicalmente o desenvolvimento. Com a adoção dos **Standalone Components** como padrão, o framework ficou mais enxuto e acessível:



Standalone Components

Componentes independentes,
sem necessidade de módulos.



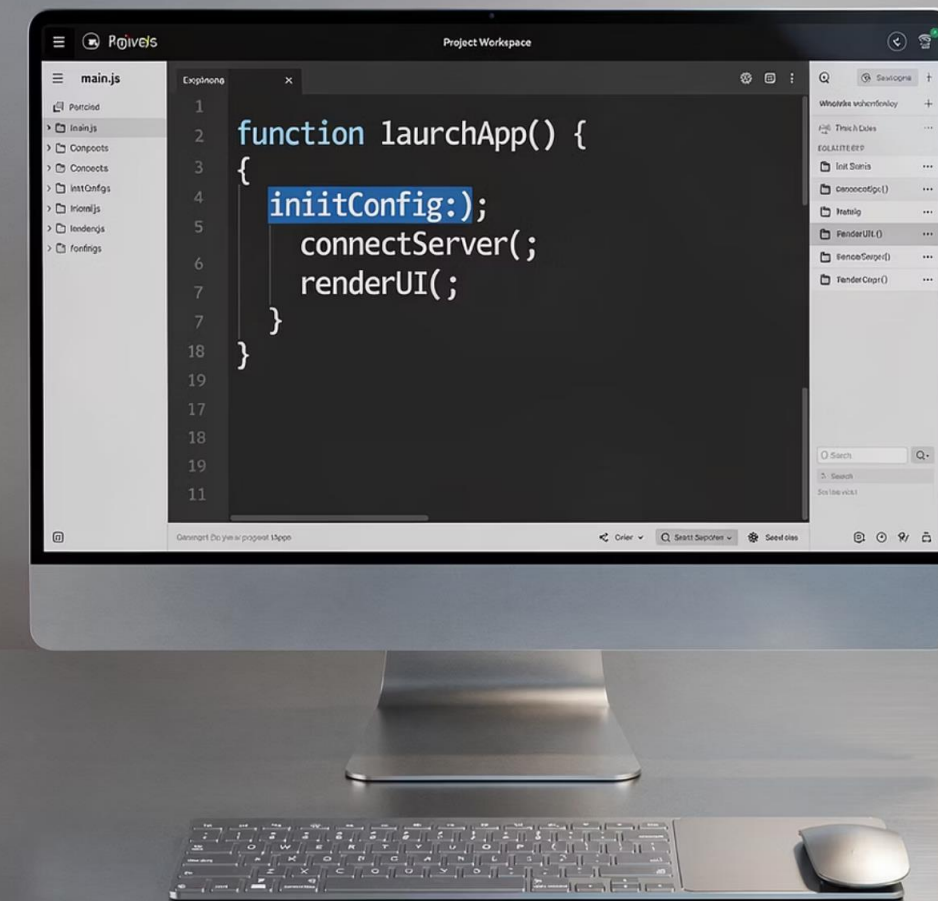
Estrutura mais simples

Menos arquivos, menos
configuração, mais foco no que
importa.



Mais produtividade

Menos tempo configurando, mais tempo desenvolvendo.



Comparação Direta — Antigo vs Moderno

Veja como a mesma tarefa é abordada de forma diferente nos dois modelos:



A diferença não é apenas estética — ela impacta diretamente a produtividade, a manutenção e a facilidade de aprendizado da plataforma.

O que são Standalone Components?

Definição

Um **Standalone Component** é um componente Angular que se autodeclara como independente. Ele não precisa ser registrado dentro de um NgModule para funcionar. Ele é, por si só, uma unidade completa e autossuficiente.

Como fica no código

```
@Component({  
  selector: 'app-produto',  
  imports: [CommonModule],  
  template: `...`  
})  
export class ProdutoComponent {}
```

- ✅ Com Standalone, cada componente declara suas próprias dependências. Isso torna o código mais explícito, mais fácil de testar e mais simples de entender.

Por que Standalone Components são importantes?

A mudança para Standalone não é apenas uma preferência de estilo — ela traz benefícios concretos, especialmente para quem está aprendendo:



Mais fácil de aprender

Sem a camada extra dos módulos, é mais fácil entender como os componentes se conectam e funcionam.



Mais fácil de navegar

Com dependências declaradas no próprio componente, fica claro o que cada parte do sistema usa.



Código mais simples

Menos arquivos e menos configuração significa menos pontos de confusão e erros desnecessários.



Padrão do mercado

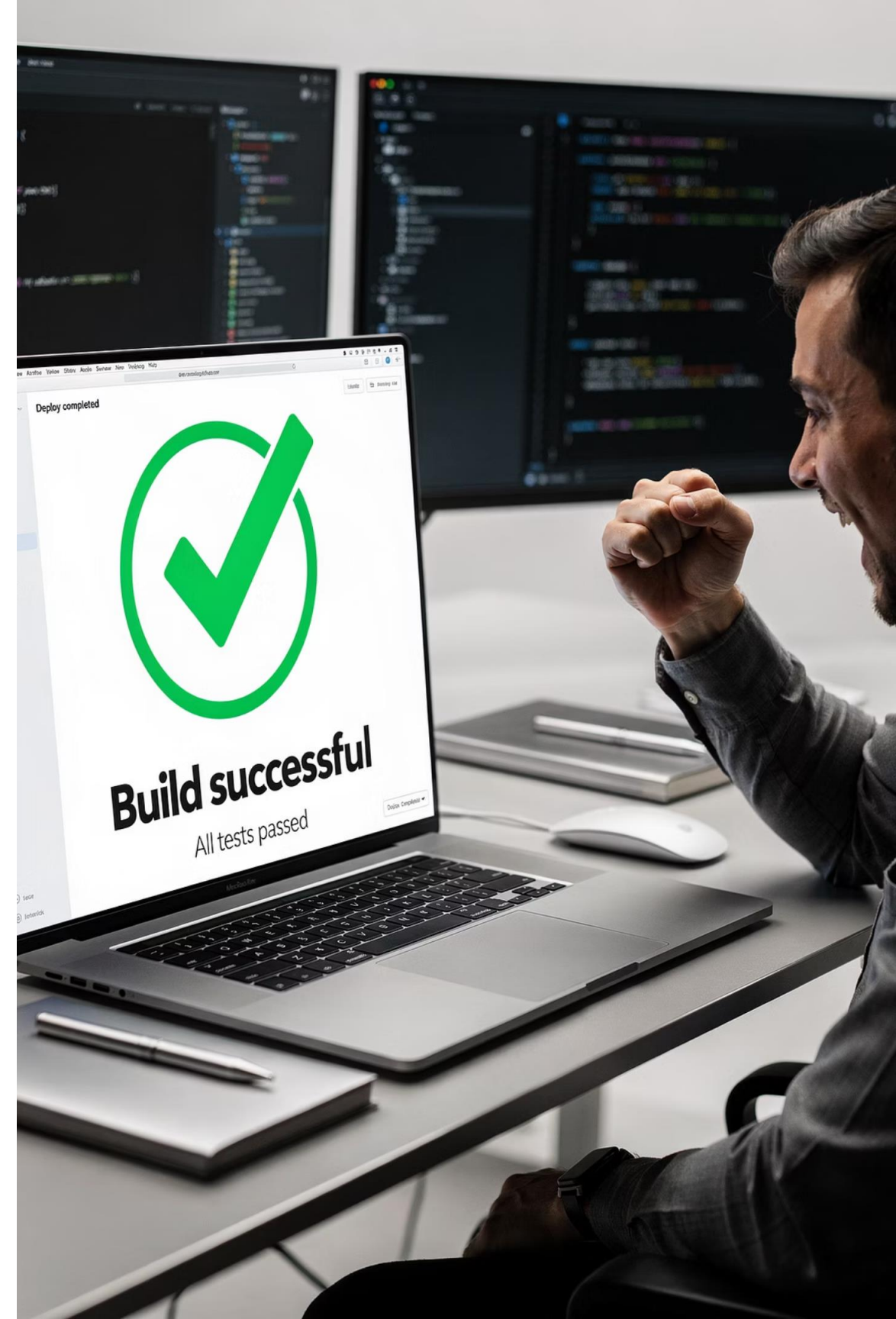
Projetos novos e vagas de emprego já exigem conhecimento do modelo Standalone. Você está aprendendo o que o mercado usa.

Você já está usando Standalone!

- ✅ O projeto que você criou na aula anterior **já utiliza Standalone Components por padrão**. Você não precisa configurar nada extra.

Quando o Angular CLI cria um novo projeto hoje, ele já adota o modelo Standalone automaticamente. Isso significa que, ao executar `ng new meu-projeto`, toda a estrutura gerada já segue a abordagem moderna.

Nas próximas aulas, você verá isso na prática ao criar e explorar seus próprios componentes.



Estrutura Baseada em Features

O que é?

O Angular moderno encoraja organizar o projeto por **funcionalidades** (features), e não apenas por tipo de arquivo.

Cada parte do sistema tem sua própria pasta com tudo que ela precisa para funcionar.

Por que isso importa?

À medida que o projeto cresce, essa organização evita o caos. Em vez de procurar um serviço em uma pasta genérica de `services/`, você encontra tudo relacionado a "produtos" dentro de `features/produtos/`.

- Facilita a manutenção
- Melhora a colaboração em equipe
- Reflete como o negócio está organizado

O que é uma Feature?

Uma **feature** (funcionalidade) é uma parte bem definida e isolada do sistema. Ela representa um domínio ou área de negócio da aplicação. Pense em cada feature como um "mini-módulo" da sua aplicação:

Produtos

Listagem, detalhes e busca de produtos da loja.

Carrinho

Adição, remoção e gerenciamento de itens no carrinho.

Login

Autenticação, cadastro e controle de sessão do usuário.

Administração

Área restrita para gerenciar produtos, pedidos e usuários.



Por que usar a estrutura por Features?

- Projeto sempre organizado
Cada funcionalidade tem seu espaço definido. Nunca haverá dúvida sobre onde um arquivo deve ficar.
- Manutenção facilitada
Alterar ou corrigir uma feature não exige vasculhar o projeto inteiro. O impacto fica contido na pasta correspondente.
- Escalabilidade real
À medida que novas funcionalidades são adicionadas, a estrutura cresce de forma natural e previsível, sem desorganização.

Estrutura Tradicional vs Estrutura Moderna

Estrutura Tradicional

Separação por **tipo de arquivo**:

```
src/app/  
├── components/  
│   ├── produto.component.ts  
│   ├── carrinho.component.ts  
│   └── login.component.ts  
├── services/  
│   ├── produto.service.ts  
│   └── auth.service.ts  
└── models/  
    └── produto.model.ts
```

O problema: para encontrar tudo relacionado a "produtos", você precisa vasculhar múltiplas pastas.

Estrutura Moderna (por Features)

Separação por **funcionalidade**:

```
src/app/  
└── features/  
    ├── produtos/  
    │   ├── produto.ts  
    │   ├── produto.service.ts  
    │   └── produto.model.ts  
    ├── carrinho/  
    └── auth/
```

Tudo sobre "produtos" está em um único lugar.
Simples e intuitivo. O termo component não aparece mais no nome dos arquivos de components.

Exemplo de Organização Moderna

Veja como ficaria a estrutura de pastas do nosso projeto no curso, seguindo a abordagem baseada em features:

Estrutura de Pastas

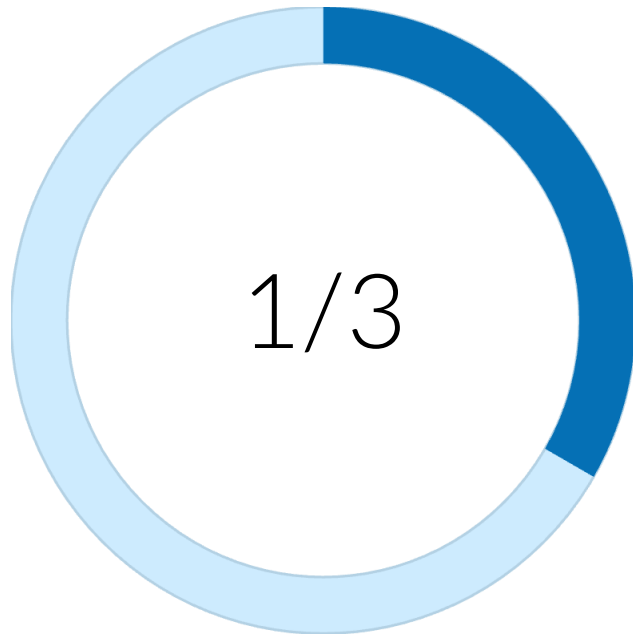
```
src/app/  
├── features/  
│   ├── produtos/  
│   │   ├── listagem/  
│   │   └── detalhe/  
│   ├── carrinho/  
│   │   └── carrinho.ts  
│   ├── checkout/  
│   │   └── checkout.ts  
│   └── admin/  
│       ├── dashboard/  
│       └── gerenciar-produtos/
```

O princípio central

Cada pasta dentro de `features/` contém **tudo** relacionado àquela funcionalidade: componentes, serviços, modelos, rotas e testes.

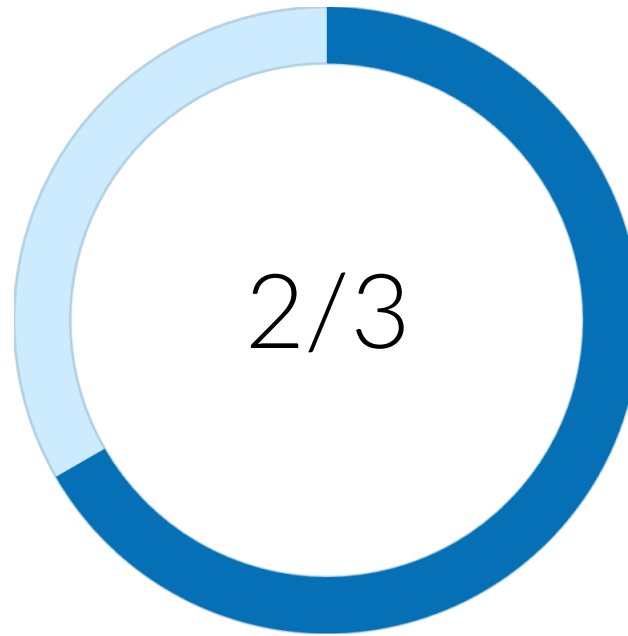
- ❏ Isso reflete diretamente a estrutura do negócio, tornando o código mais intuitivo para qualquer desenvolvedor que entre no projeto.

Vantagens da Organização por Features



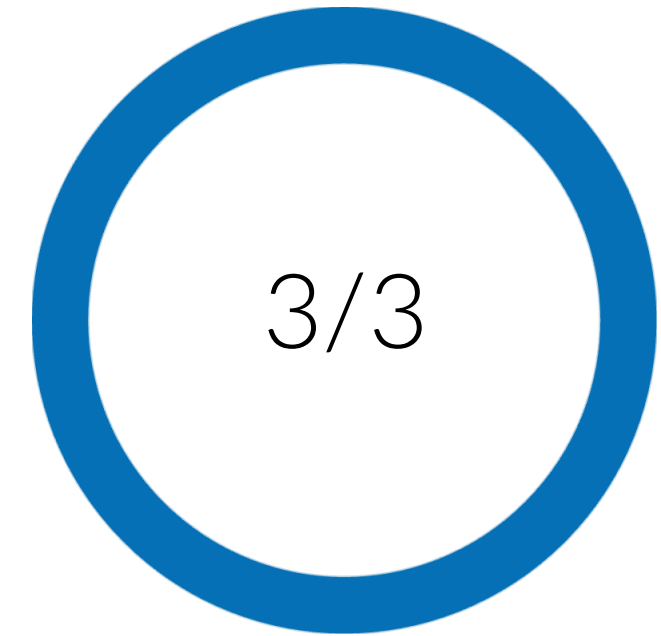
Mais organizado

Código estruturado por domínio de negócio é sempre mais fácil de navegar e entender.



Pronto para crescer

A estrutura por features escala naturalmente conforme novas funcionalidades são adicionadas ao projeto.



Padrão do mercado

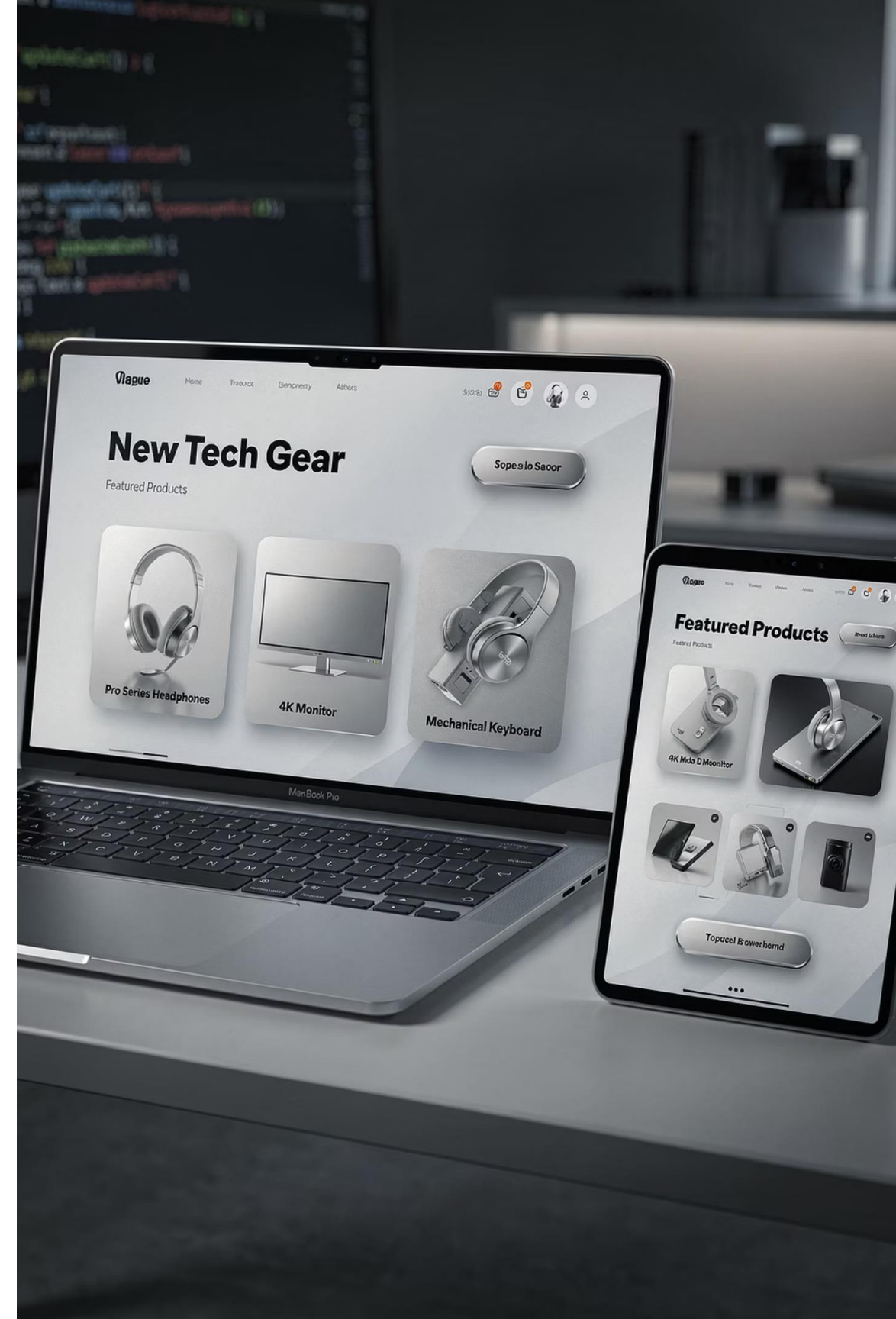
Empresas de médio e grande porte adotam essa arquitetura. Aprender agora é um diferencial real.

O Projeto do Curso: Um E-commerce Completo

Durante este curso, você não vai apenas aprender conceitos isolados — você vai **construir uma aplicação real** do zero até a produção.

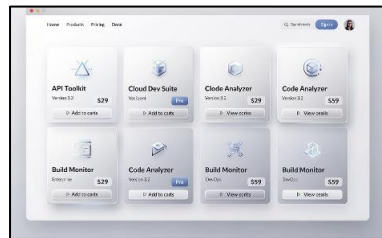
O projeto escolhido é um **e-commerce**: uma das aplicações mais completas e representativas do mundo real, que envolve desde listagem de produtos até área administrativa.

- ❗ Construir um projeto completo é a melhor forma de consolidar o aprendizado e ter algo concreto para mostrar no portfólio.



Funcionalidades do Projeto

O e-commerce que construiremos terá todas as funcionalidades essenciais de uma aplicação profissional:



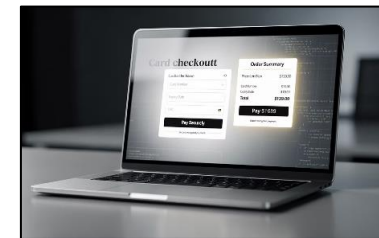
Listagem de Produtos

Exibição de produtos com filtros, busca e navegação por categorias.



Carrinho de Compras

Adicionar, remover e gerenciar produtos antes de finalizar a compra.



Checkout

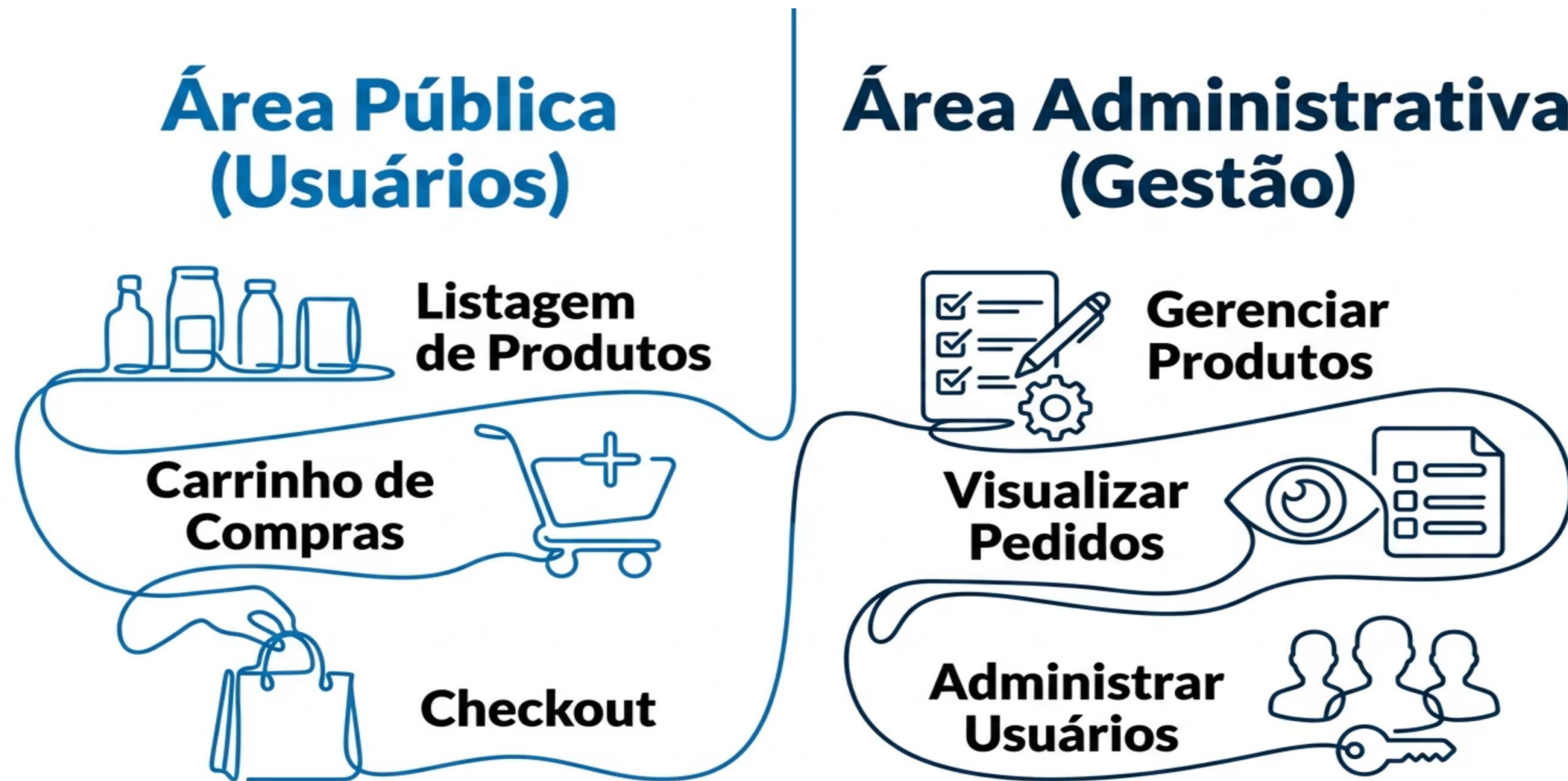
Fluxo completo de finalização de pedido com dados do cliente.



Área Administrativa

Painel para gerenciar produtos, pedidos e usuários da plataforma.

Estrutura do Projeto – Duas Grandes Áreas



Essa divisão clara entre área pública e administrativa é um padrão comum em aplicações web profissionais e ajudará a entender conceitos como **guardas de rota** e **autenticação** mais à frente no curso.

A Aplicação Será Construída Passo a Passo

Você **não vai construir tudo de uma vez**. Cada aula adiciona uma peça ao projeto, e ao final do curso, você terá um e-commerce completo e funcional.

1

Fundamentos

Componentes, rotas e serviços básicos

2

Features Públicas

Produtos, carrinho e checkout

3

Features Admin

Dashboard e gerenciamento

4

App Completo

Integração e refinamentos finais



O que você precisa entender agora

Neste momento do curso, o importante é absorver o panorama geral.

Você não precisa decorar nada — apenas compreender as três ideias centrais:

Angular moderno é mais simples

A plataforma evoluiu para remover complexidade desnecessária. O código que você vai escrever é mais enxuto e legível do que nas versões antigas.

Usamos Standalone Components

Componentes independentes são o novo padrão. Sem NgModules obrigatórios, sem configuração extra — apenas componentes que se autodeclaram.

Organização por features

O projeto é estruturado por funcionalidade, não por tipo de arquivo. Isso torna o código mais intuitivo e fácil de navegar.

O que vem a seguir

Agora que você tem o mapa do território, é hora de colocar a mão no código. Nas próximas aulas, o aprendizado será totalmente prático:

1

Criar Componentes

Aprender a criar e estruturar componentes Angular com o CLI.

2

Construir a Interface

Começar a montar as telas do e-commerce com HTML e Angular.

3

Trabalhar na Aplicação Real

Aplicar os conceitos diretamente no projeto do curso, feature por feature.



Resumo da Aula

Veja o que você aprendeu nesta aula:

1

Angular antigo vs moderno

O modelo antigo usava NgModules obrigatórios. O moderno adota Standalone Components como padrão, tornando tudo mais simples.

3

Organização por features

O projeto é dividido por funcionalidade (produtos, carrinho, admin), não por tipo de arquivo. Isso facilita manutenção e escalabilidade.

2

Standalone Components

Componentes independentes que não precisam ser declarados em módulos. Já é o padrão no seu projeto criado na aula anterior.

4

Visão geral do projeto

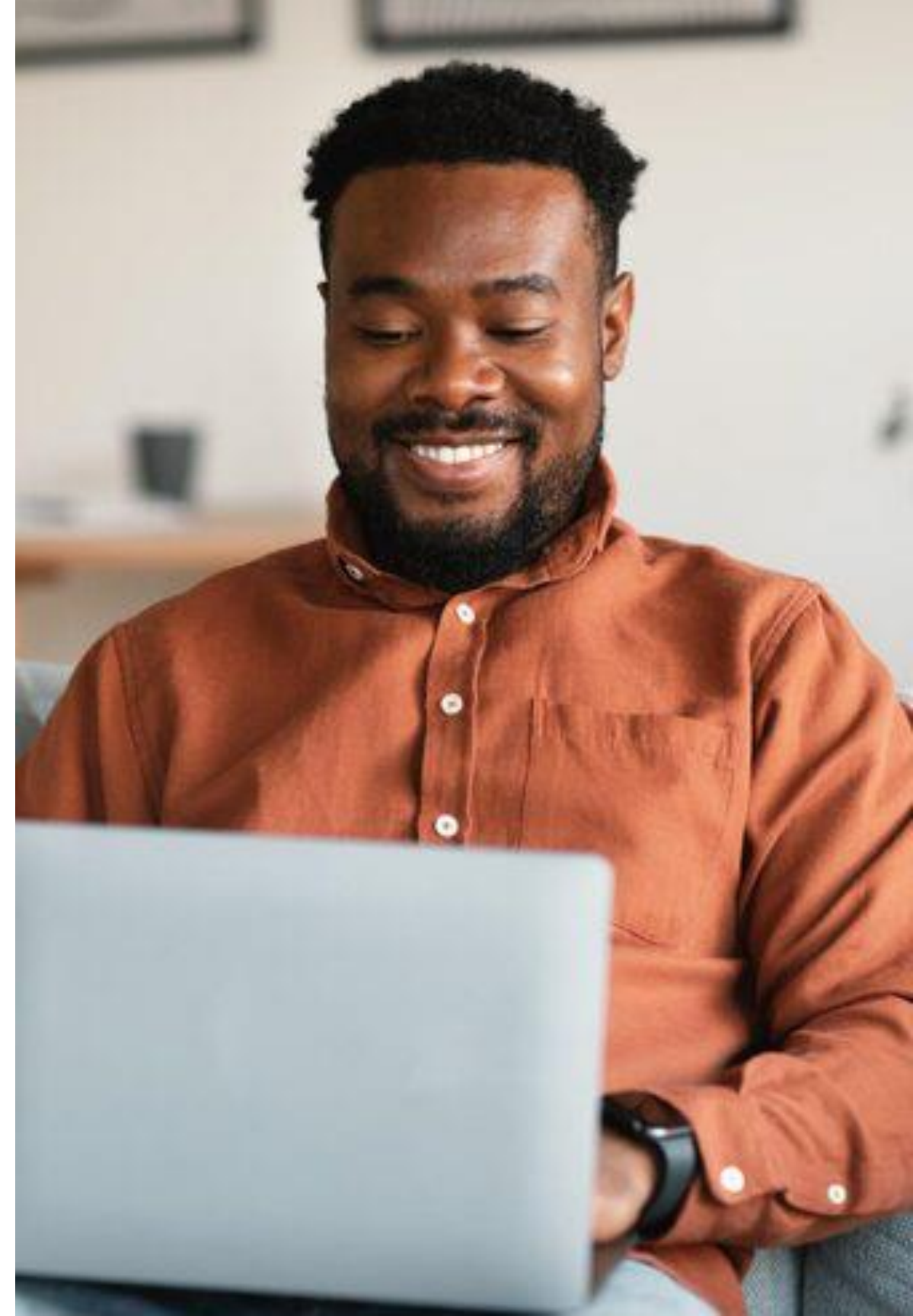
Você vai construir um e-commerce completo, com área pública e administrativa, passo a passo ao longo do curso.

Parabéns por chegar até aqui!

Agora você já entende **como o Angular moderno funciona**, por que ele evoluiu dessa forma e como seu projeto será estruturado. Esse contexto vai facilitar muito o aprendizado nas próximas aulas.

Você não apenas aprendeu conceitos — você criou a base mental necessária para entender cada decisão que tomaremos juntos ao longo do curso. Cada linha de código que você escrever a partir de agora fará muito mais sentido.

- ✔ Continue com confiança.
O mais difícil — a visão geral — já está com você.



Dúvidas

?

